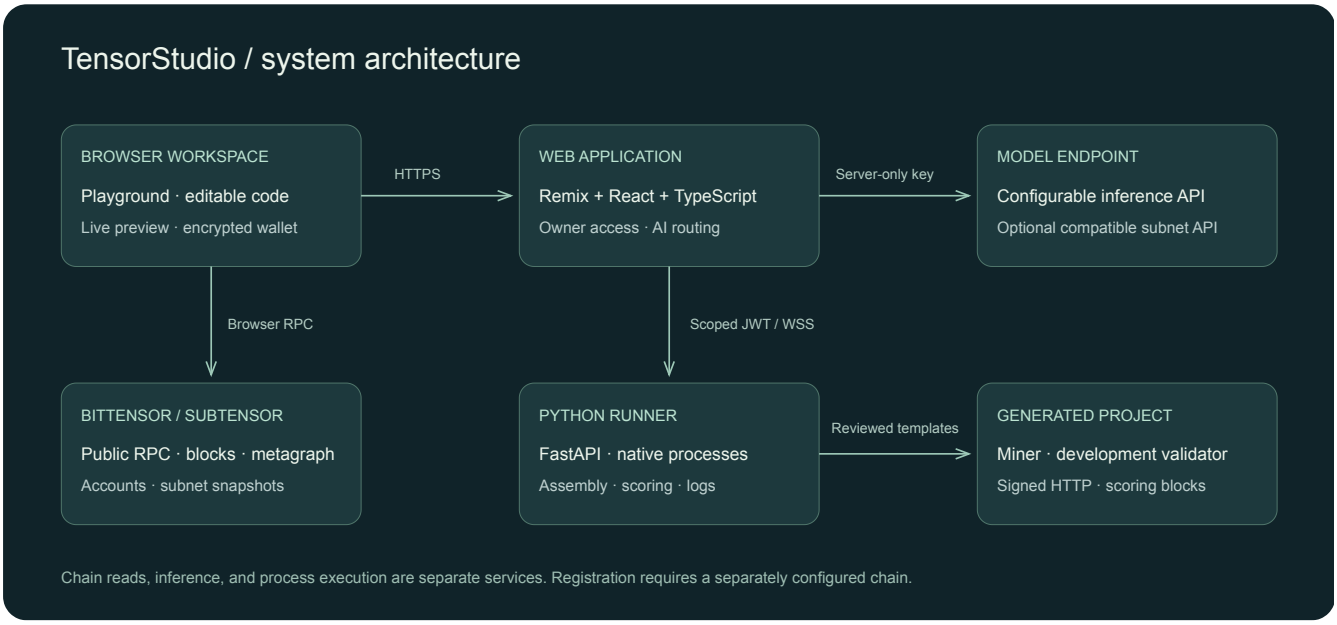
TENSORSTUDIO / ARCHITECTURE GUIDE

How the workspace fits together.



Three independent flows

1. **Assistant and generation:** browser → validated web action → configured model endpoint → structured suggestions or code.
2. **Project execution:** authenticated browser session → scoped runner token → FastAPI → assembly, fixtures or native process output.
3. **Network observation:** browser RPC → Subtensor → block-pinned views of blocks, events, accounts and subnet state.

Connecting to a public chain does not register a generated miner. Configuring inference does not prove a TensorStudio subnet serves it.

Architecture reflects the repository's Remix, React, FastAPI, WebContainer and Polkadot.js integration.

COMPONENTS AND DATA

Inside the system.

Browser workspace

React renders the guided playground, editor, preview, process views, wallet and explorer. WebContainers run browser projects. Wallet material is encrypted in browser-local storage. Explorer history is a bounded recent index.

Web application

Remix actions validate message shapes and requested model selections. Provider credentials are server-only. The hosted owner workspace uses access protection; execution is not a public multi-tenant service.

Python runner

FastAPI validates project requests, assembles fixed reviewed templates and runs code/scoring checks. It manages native processes with bounded lifetime/output and streams console events over WebSocket connections.

Generated project

Recipe, dataset, miner, model client, development validator, scoring blocks and checks are inspectable files. A model supplies miner answers rather than a dataset lookup. Signed HTTP separates authorized request identities from inference.

Subtensor integration

Runtime-aware RPC reads provide best/finalized heads, decoded calls/events, account views and subnet snapshots. The Python SDK supports generated request authentication and development-chain workflows.

TRUST AND OPERATION

Keep the boundaries clear.

Boundary	Design
Browser / web server	Owner access for operational pages; provider keys stay server-side.
Web / hosted runner	Short-lived signed sessions, exact origin checks, TLS and authenticated WebSockets.
Runner / projects	Reviewed assembly templates, input validation, controlled paths, process limits. Trusted code execution, not an OS sandbox.
Miner / validator	Hotkey-signed HTTP requests and validator allowlists.
Project / chain	Read-only public observation is distinct from registration, serving, weights and rewards.

Deployment

The web app is deployed on Vercel and the runner on Render Free. The runner may sleep and has ephemeral local files. A production service needs durable storage, recovery, monitoring, quota controls and tenant isolation before accepting untrusted multi-user workloads.

Development and verification

Install dependencies with the root pnpm lockfile and runner uv lockfile. Run the local setup script to pair runner credentials. Use TypeScript checks, frontend tests, runner tests and a production build. Browser checks should exercise inference, preview, assembly and WebSocket completion separately.

Primary references

Bittensor network
Signed requests
Polkadot.js API
Render free-service limits